Allsky GO

Installation Manual [Gen2] v1.0

Oct 2026

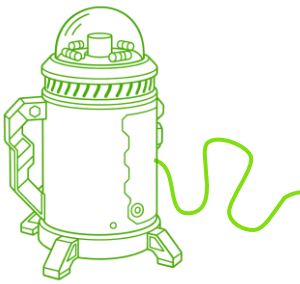
- Raspberry Pi4
- DS3231 RTC Module
- GT-U7 GPS Module
- SHT31 Humidity and Temperature Sensor
- Button Press Scripts and Services













Contents

INITIAL RASPBERRY SET-UP 3

Real-Time Clock (RTC) MODULE 5

Enabling I^2C 6

SENSOR INSTALLATION 8

Installing the GPS Module 8

Installing the Humidity Sensor (SHT31) **without** RTC Board for Pi4 9

Installing the Humidity Sensor (SHT31) **with** RTC Board for Pi4 10

SCRIPTS 14

Step1 : Loading Custom Scripts for Allsky GO from Terminal 14

Step2. Button Press Service [button_script.service] 15

Step3. Start-up Buzzer Service 16

Debugging System Services 17

RTC & TIME SYNCHRONISATION 18

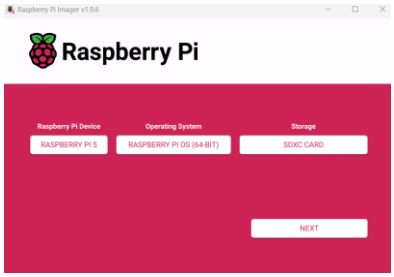
ALLSKY SETTINGS 20

DEBUGGING AND TROUBLE SHOOTING 21



INITIAL RASPBERRY SET-UP

Use Raspberry PI Imager to Install Bookworm onto an SD Card according to your Raspberry PI. *(The desktop version is recommended to facilitate system file access through remote desktop)*



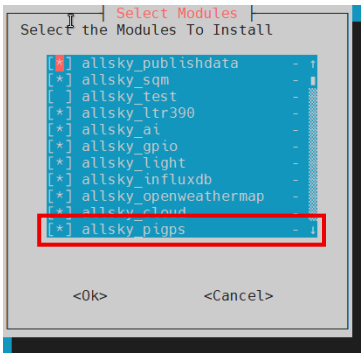
Use the custom setting to personalize your WIFI connection, passwords and IP Address (ie *allskygo.local).*

*Tip: Referring your Wifi connection to a **Mobile Phone Hotspot** will enable automatic access to your mobile device when out in the field.*

Secondly, follow steps to install the latest distribution of the **Allsky Team Software Package** from their Github for GPS and SHT31 you will also need to follow their instructions for the following **Allsky Modules**:

- **allsky_pigps**
- **allsky_dewheater**

Their modules will manage the GPS Sensor and SHT31 Sensor – allowing you to integrate their data output in your allsky settings and overlay variables.



🎯 Optimizing Raspberry PI Boot Configuration for Minimal power consumption

Modify Boot configuration with the following command:

```
sudo nano /boot/firmware/config.txt
```

Add the following lines

```
dtoverlay=i2c-rtc,ds3231
#Required for Pi4 RTC with DS3231

dtoverlay=pwm-2chan
#Enables dual-channel PWM (Pulse Width Modulation), used to drive the buzzer.

hdmi_blanking=2
#Disables HDMI output, preventing unnecessary power usage if no display is connected. Perfect for
#off-grid or headless operation.

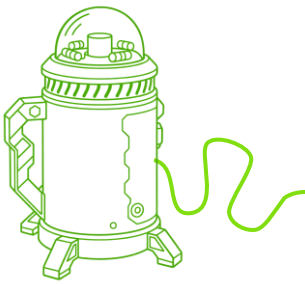
dtoverlay=disable-bt
#Disables onboard Bluetooth, freeing system resources and reducing power draw—smart move if you do
#not need wireless peripherals.

dtparam=audio=off
#Turns off the onboard sound system, specifically the snd_bcm2835 driver. This prevents the audio
#hardware from loading—again, saving resources.

# Disable ACT LED (green)
dtparam=act_led_trigger=none
dtparam=act_led_activelow=off

# Disable PWR LED
dtparam=pwr_led_trigger=none
dtparam=pwr_led_activelow=off
```

To Exit - Press **`Ctrl + X`** & **`Yes`** to Save



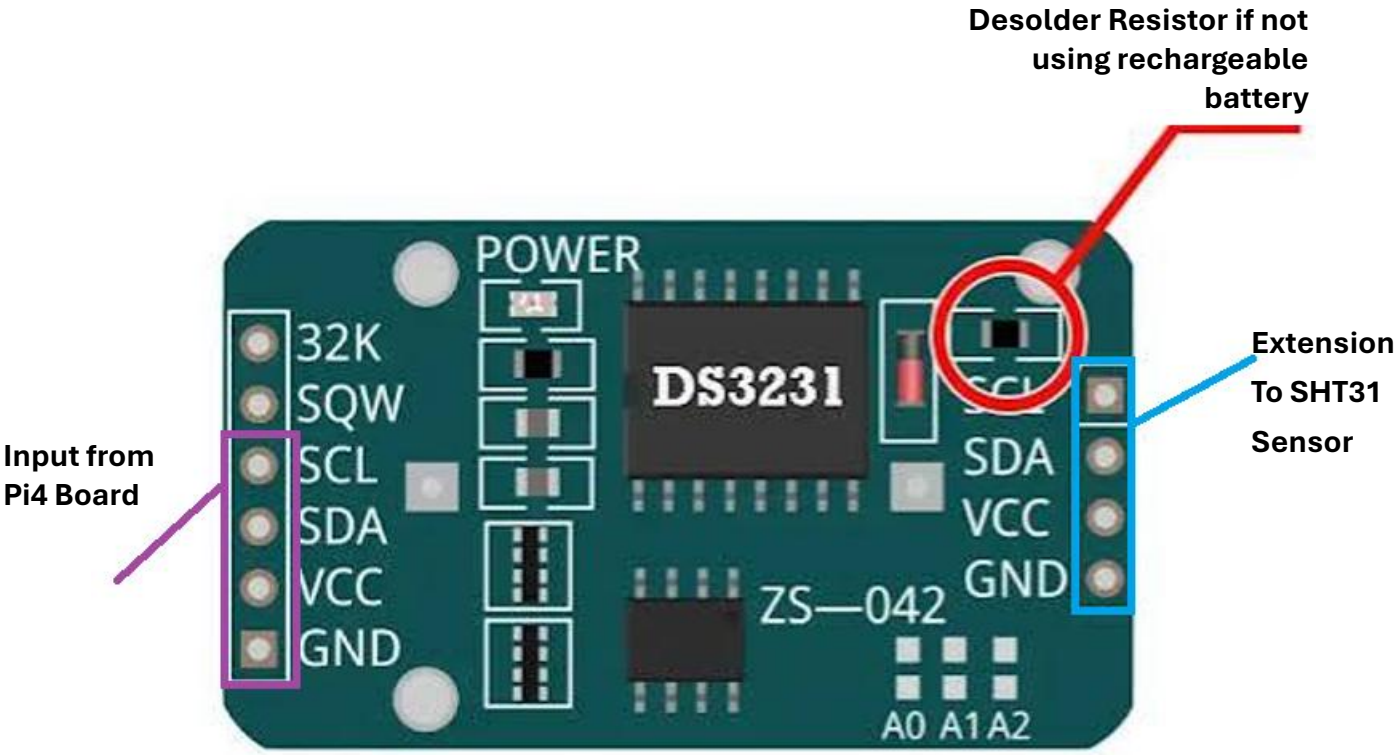


Real-Time Clock (RTC) MODULE

Set-up RTC with **DS3231 for Raspberry Pi**

A **DS3231 RTC (Real-Time Clock)** is a great addition to a Pi 4 if you want it to keep accurate time when powered off and without internet access. The coin battery keeps track of time when the system is powered down.

On the board - 4 of the 6 Pins are used (VCC,GND,SDA,SCL) You can leave **SQW** and **32K** completely unconnected. If using a standard CR2032 battery – **desolder the resistor** in the top right corner. The SHT31 sensor can be **daisy-chained** in series connecting to the output connections on the DS3231.



SENSOR INSTALLATION

Enabling I²C

From Terminal run:

```
sudo raspi-config
```

Then Select in the pop-up

└Interface Options

└ I2C

└ Enable

Exit and Reboot:

```
sudo reboot
```

Check if the RTC is detected

Install I²C tools:

```
sudo apt update
```

```
sudo apt install -y i2c-tools
sudo i2cdetect -y 1
```

You should see address 0x68.

```
pi@allskygo:~ $ sudo i2cdetect -y 1
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- 57 -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- 68 -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

Check the correct detection of the DS3231

```
dmesg | grep rtc
```

You should see the DS3231 being detected.

Set the RTC time

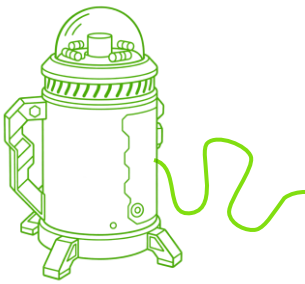
After the Pi has the correct time:

```
sudo hwclock -w
```

This writes the system time to the DS3231.

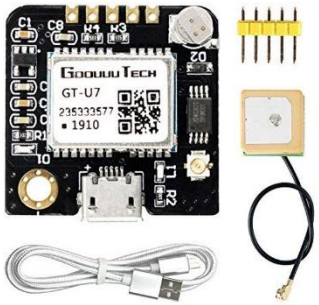
Verify, with read

```
sudo hwclock -r
```





Installing the GPS Module (GT-U7)



#USB Connection & Installation of GT-U7 GPS Module

The Allsky module should do most of the work.

The installation can be checked with the commands below:

****

#Identify USB Serial Port - check connection to USB listed as U-Blox:

```
lsusb
```

#List connected Devices

#Look for /dev/ttyUSB0 or /dev/ttyACM0 - this is your GPS Port:

```
dmesg | grep tty
```

#Check raw data feed - If you see NMEA sentences (lines starting with $GPGGA, $GPRMC), your GPS is working (Ctrl-C to Exit)

```
cat /dev/ttyACM0
```

#Verify if Running

```
sudo systemctl status gpsd
```

#If not active, restart with

```
sudo systemctl restart gpsd
```

Installing the Humidity Sensor (SHT31) **without** RTC Board for Pi4

IF RTC Module has already been installed – skip this step

****

The allsky module will do most of the work, but you may need to **enable I²C on the Pi**
Check using the command line (non-interactive)

```
sudo raspi-config nonint get_i2c
```

Output 0 means **enabled,** Output 1 means **disabled**

Use raspi-config to enable I²C:

```
sudo raspi-config
```

Navigate to *Interfacing Options → I2C → Enable*.

The SHT31 sensor typically communicates via the **I²C interface**, which **expects to use specific GPIO pins**)

- **GPIO2 (Pin 3)** for SDA
- **GPIO3 (Pin 5)** for SCL

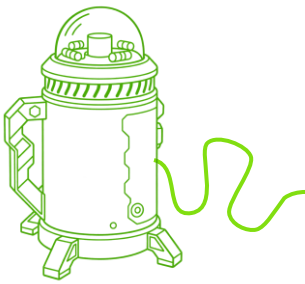
✅ 3. Check for I2C device node, Run:

```
ls /dev/i2c-1
```

If you see /dev/i2c-1, I2C is enabled. If you get "No such file or directory," it is not

✅ 4. Scan for connected I2C devices, install tools if needed:

```
sudo apt install -y i2c-tools
```





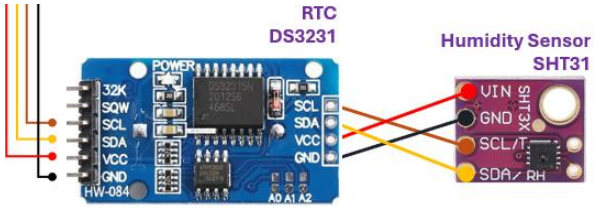
Then scan:

```
sudo i2cdetect -y 1
```

You should see a grid with addresses like 0x44 (or 0x45) if your SHT31 sensor is connected and working!

```
pi@allskygo2:~ $ sudo i2cdetect -y 1
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- 44 -- -- -- -- -- -- -- -- -- -- --
```

Installing the Humidity Sensor (SHT31) **with** RTC Board for Pi4



A) Scan for Connected I2C Devices / DAISY CHAINED TO RTC BOARD

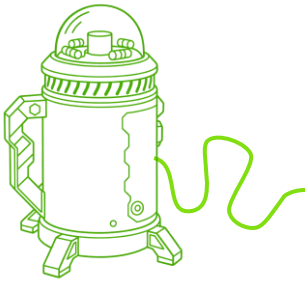
Following installation of RTC and daisy connection to VCC,GND,SDA,SCL

✅ 4. Scan for connected I2C devices, install tools if needed:

```
sudo apt install -y i2c-tools
```

Then scan:

```
sudo i2cdetect -y 1
```





You should see a grid with addresses like 0x44 (or 0x45) if your SHT31 sensor is connected and working.

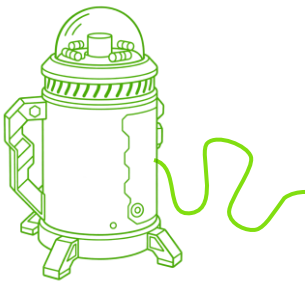
```
pi@allskygo:~ $ sudo i2cdetect -y 1
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- 44 -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- 57 -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- 68 -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

Address	Device
0x44	SHT31 temperature/humidity sensor
0x57	AT24C32 EEPROM (commonly present on DS3231 modules)
0x68	DS3231 RTC
UU at 0x68	A kernel driver has already claimed the RTC

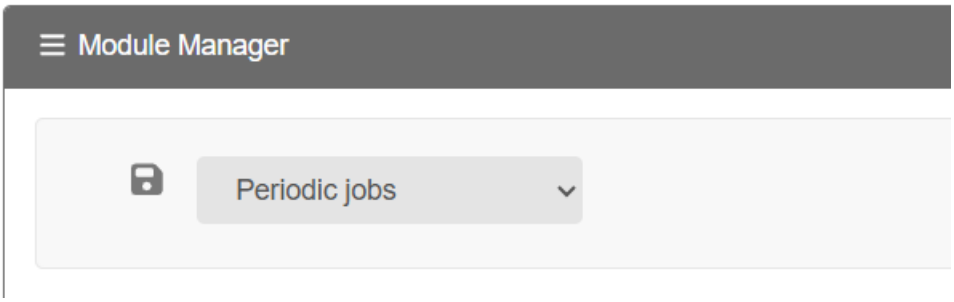
The UU is normal and actually a good sign. It means a driver (typically rtc-ds1307 for the DS3231 family) has bound to address 0x68, so i2cdetect will not probe it directly.

✅ 5. **Dew Heater Module Set-up**

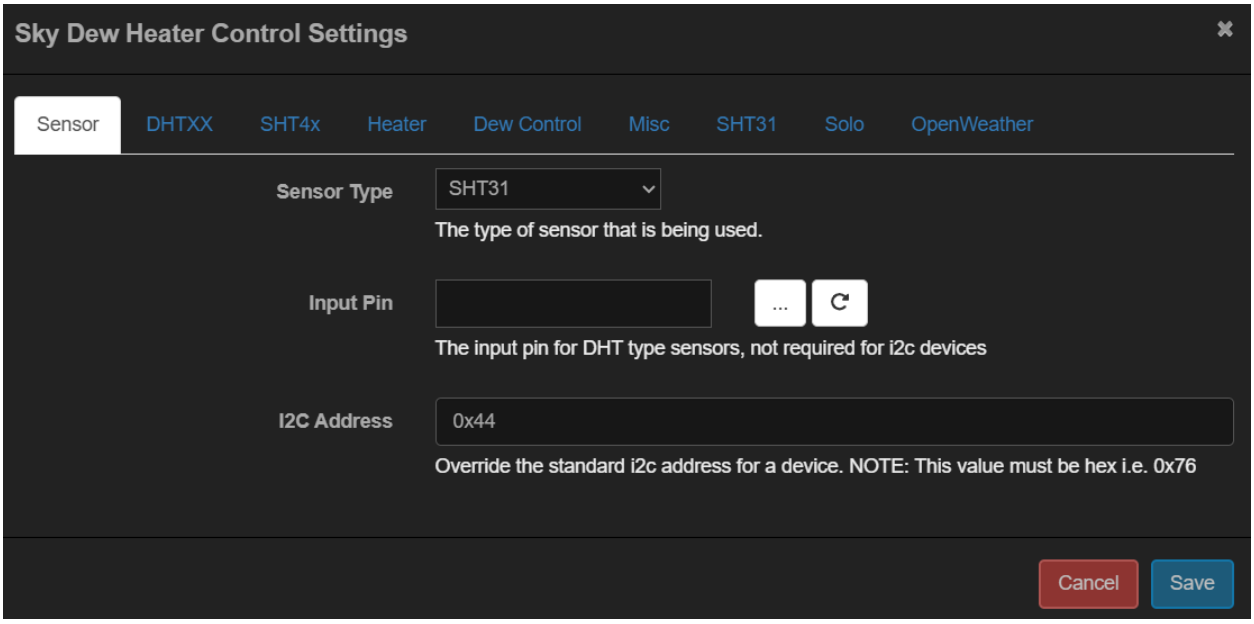
Enable the Dew Heater Module in Allsky under Periodic Jobs



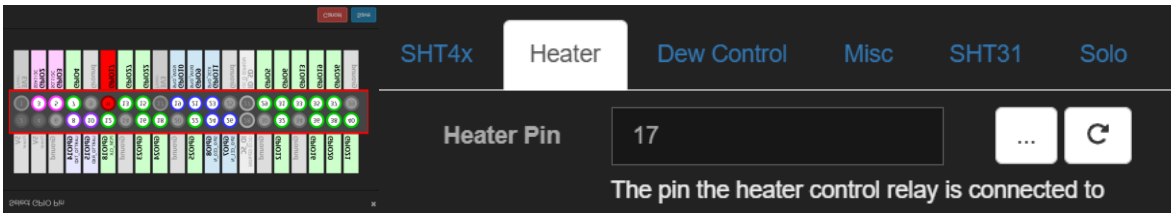


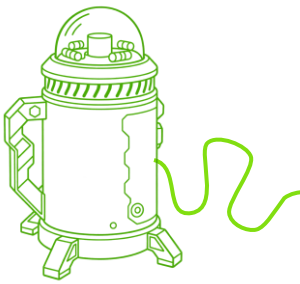


Add this 0x44 to the Dew Heater Control Module Setting:



** IMPORTANT : even if you don't have a dew heater connected, you still need to assign a GPIO pin in the "Heater" Tab for the module to work correctly.

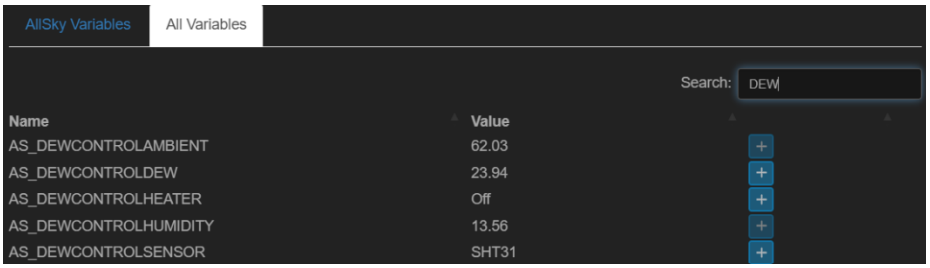






After installation – **reboot your Allsky** : and the variables should appear in the Overlay Section – “All Variables” to add to your overlay options.

Set as Number, Set Text Format {:.2f}



SCRIPTS

Step1 : Loading Custom Scripts for Allsky GO from Terminal

#Create Folder for Custom Scripts on Raspberry Terminal (*assumed Username = pi)

```
mkdir /home/pi/scripts
```

#Move to Folder

```
cd /home/pi/scripts
```

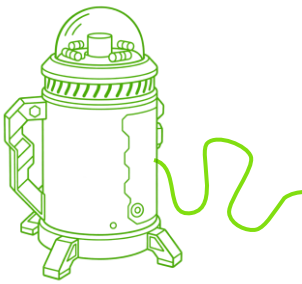
List of Scripts to copy from Github;

- button_a+b.py — Controls Button Press Actions
- Buzzer Melodies(7) — Subprocesses to drive Buzzer at Button Press
- wps_connection.py — Subprocess for WPS connection after Button Press

Copy into your /scripts folder:

1. #Create Nano File for each script in the scripts folder: i.e., `nano button_a+b.py`
2. **Paste** the text from github files into nano text editor
3. **Exit** - Press **Ctrl + X** & **Yes** to Save – preserving the python file names.

Note: connected hardware can be tested with some python script commands in the Test Folder
Note: Python Scripts and Processes based around Raspberry Pi4 Bookworm and specific GPIO Pin usage. Read the code to check any references that may need to be updated for your specific set-up.





Step2. Button Press Service [button_script.service]

✅ Step-by-Step: Create a systemd service

```
sudo nano /etc/systemd/system/button_script.service
```

Paste into: /etc/systemd/system/button_script.service the following text;

```
[Unit]

Description=Button Script for Allsky
After=network.target

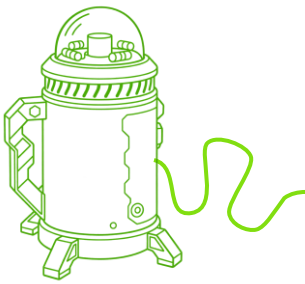
[Service]
ExecStart=/usr/bin/python3 /home/pi/scripts/button_a+b.py
WorkingDirectory=/home/pi/scripts
StandardOutput=inherit
StandardError=inherit
Restart=always
User=pi
#CHANGE TO YOUR USER IF REQUIRED
[Install]
WantedBy=multi-user.target
```

RESTART Service and Daemon

```
sudo systemctl restart button_script.service

sudo systemctl enable button_script.service

sudo systemctl daemon-reload
```





Step3. Start-up Buzzer Service

✅ Step-by-Step: Create a systemd service

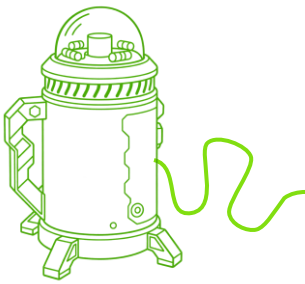
```
sudo nano /etc/systemd/system/startup_buzzer.service
```

Paste into: /etc/systemd/system/startup_buzzer.service the following text;

```
[Unit]
Description=Startup Buzzer Script
After=multi-user.target sysinit.target local-fs.target
Wants=dev-gpiomem
[Service]
Type=oneshot
ExecStart=/usr/bin/python3 /home/pi/scripts/startup_buzzer.py
WorkingDirectory=/home/pi/scripts
User=pi
#CHANGE TO YOUR USER IF REQUIRED
Restart=no
[Install]
WantedBy=multi-user.target
```

RESTART Service and Daemon

```
sudo systemctl restart startup_buzzer.service
sudo systemctl enable startup_buzzer.service
sudo systemctl daemon-reload
```





Debugging System Services

Check its status:

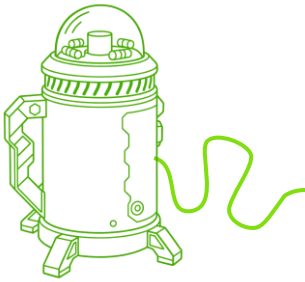
```
sudo systemctl status button_script.service
```

Check the Debug Log

```
journalctl -u button_script.service
```

Check all Running Processes & Resource Level

```
htop
```





RTC & TIME SYNCHRONISATION

In an Allsky camera setup — where capturing accurate night sky images depends on knowing exactly when day turns to night — keeping your Raspberry Pi's system clock in sync is essential.

🧠 What Is RTC?

RTC stands for **Real-Time Clock** — a tiny onboard chip that keeps track of time even when the system is powered off. On Raspberry Pi 5, the RTC is built into the power management IC (PMIC), but it needs a **battery backup** to retain time during shutdowns or power loss.

Without an RTC or external time source, the Pi boots up thinking it is 1970 — not ideal for astrophotography!

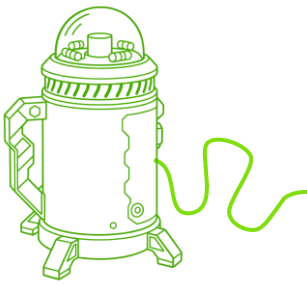
🌓 Why Time Sync Matters for Allsky

Allsky software uses timestamps to:

- Trigger **nighttime image capture**
- Generate **timelapse videos**, **keograms**, and **startrails**
- Overlay **date/time/location** on images

If your Pi does not know the correct time, it might start imaging during daylight or miss celestial events entirely.

🛠️ Three Ways to Keep Time Accurate



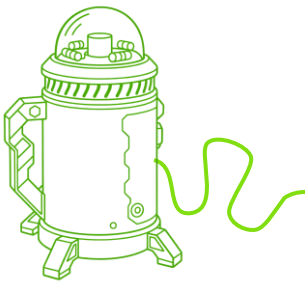


Option	Pros ✅	Cons ❌
🅐 **Built-in RTC + External Battery** 🔋	- Keeps time during shutdown or power loss - No dependency on network or GPS - Simple and low power	- Accuracy can drift over time - Requires correct battery and config - Doesn't auto-correct or sync
🅑 **Network NTP Servers** 🌐	- Highly accurate with low offset - Easy to set up with Pi OS - Can auto-sync continuously	- Needs stable internet connection - Not ideal for remote observatories - Vulnerable to connectivity drops
🅒 **GPS Module (GT-U7)** 🛰️	- Atomic-level precision - Works offline, great for remote locations - Provides both date and time from satellites	- Requires clear sky view - Longer Time-To-First-Fix without A-GPS - Initial setup is more complex

The GPS module is an excellent solution for Off-Grid time keeping but is heavily dependent on a **valid GPS Fix** ; without which no timing synchronization can be obtained. The Built in RTC with external battery is recommended / necessary as primary back-up.

For a robust Allsky setup:

A) Internet available? > Sync from NTP > Update DS3231
B) No Internet? > Read DS3231
C) GPS FIX available? > Optionally correct system time--> Update DS3231





ALLSKY SETTINGS

🌞🌒 Image Capture Settings

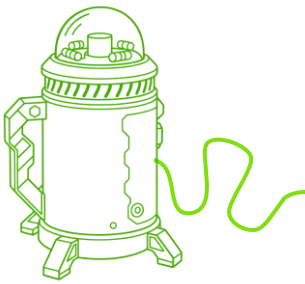
Setting	Status / Value	Function
Save Day/Night Images	✅ Enabled	Ensure image capture continues even if time zone data is missing.
Auto Exposure (Day/Night)	✅ Enabled	Adjusts light levels automatically for consistent image quality.
Max Exposure Duration	30 seconds	Prevents errors during day/night transition misidentification.

🌠 Startrail Configuration

Setting	Value / Status	Function
Startrail Threshold	0.3	Raised from default 0.1 to tolerate foreground elements.
Generate Visuals	❌ Disabled	Startrail, Timelapse, Keogram generation off to conserve battery. Activated manually through Button press.

🌎 Web Upload

Setting	Status	Function
Upload Images	❌ Disabled	Disables sync attempts—ideal for off-grid use and testing.





DEBUGGING AND TROUBLE SHOOTING

Raspberry Shuts down when connecting USB cable to charge Power bank When connecting external cable to the USB-C Port, the Power bank interrupts the supply to the raspberry Pi; enough to shut down the PI. The external USB port is not hot swappable. Ensure Raspberry PI is shutdown before connecting Power bank Charger. The Charger can be charged as it powers the raspberry pi – but is not recommended, may shorten battery life.

Functions within Scripts do not work Scripts developed with Raspberry Pi4. Hardware or OS changes in the Pi may affect legacy script compatibility (e.g., GPIO mapping, overlay handling) if using previous version.

Power Button Light does not Turn On when pressed. Power bank is flat (0%)

***GPS No Fix* showing in Overlay.** Please ensure the camera has direct access to the sky on all four sides. Large buildings or trees may hinder the visibility of Satellites to obtain reliable fix. The GPS Service will continue looking for a valid fix.

GPS False Fix. Unfortunately, the GT-U7 module can lock onto a false fix; it thinks it is connected but resulting Coordinates at 0:0 or negative altitude.

From Terminal

- **Service Debugging**; *sudo journalctl -u [SERVICE NAME]*
- **Check Running Processes**: htop

